

Guia de Criação de Projetos

Fluxo padrão para preparar novos projetos até o início da implementação

ANTIGRAVITY KIT	DOCUMENTAÇÃO • AMBIENTE • ESPECIFICAÇÃO
-----------------	--

Documento operacional • Versão 1.0

Objetivo

Este guia define o processo para iniciar um novo projeto até o ponto em que o desenvolvimento pode começar. O Antigravity Kit é responsável por inicializar o repositório, a aplicação e toda a estrutura operacional e documental necessária.

Antes da implementação, a responsabilidade humana é criar a pasta inicial, preparar os arquivos de diretiva, configurar o ambiente, fornecer o token GitHub com segurança, executar o Antigravity Kit, gerar e revisar as especificações definitivas de projeto e design.

Responsabilidades

Responsável	Atividades
Pessoa responsável pelo projeto	Criar pasta, configurar <code>.env</code> , registrar ideia, executar prompts, revisar e aprovar escopo
Antigravity Kit	Inicializar Git, aplicação, estrutura de diretórios, arquivos operacionais, padrões de código e ferramentas do projeto
IA de especificação	Transformar as ideias e instruções em <code>5.projeto.md</code> e <code>5.design.md</code>
CI/CD e provedores	Armazenar secrets remotos, executar verificações e realizar deploy quando configurado

Princípios

- O código não começa antes das fontes de verdade serem revisadas.
- O Antigravity Kit cria a estrutura técnica; este guia não a recria manualmente.
- A documentação oficial permanece em `documentation/`.
- As diretivas de elaboração ficam em `documentation/directives/`.
- Tokens, chaves e credenciais nunca são versionados.
- O GitHub Personal Access Token é mantido exclusivamente no `.env` local ou em secrets remotos autorizados.
- O MVP é separado de evoluções e funcionalidades futuras.
- O projeto começa somente após o checkpoint de prontidão.

Pré-requisitos

Antes de iniciar:

- Node.js LTS instalado.
- Gerenciador de pacotes disponível: `npm`, `pnpm` ou `yarn`.
- Terminal disponível.
- GitHub Personal Access Token criado com permissões mínimas necessárias.

- Acesso aos serviços externos previstos, quando aplicável.
- Antigravity Kit disponível no ambiente de desenvolvimento.
- Templates organizacionais dos prompts e instruções disponíveis.

Comandos de verificação:

Código de referência

```
node --version
npm --version
git --version
```

Visão geral

Código de referência

Criar pasta → criar directives/design → criar diretivas vazias → criar .env
→ configurar GitHub PAT → executar ag-kit init → revisar estrutura
→ preencher ideia → gerar projeto → gerar design → revisar → iniciar desenvolvimento

Etapa 1 — Criar a pasta inicial

Crie a pasta raiz do novo projeto e entre nela.

Código de referência

```
mkdir nome-do-projeto
cd nome-do-projeto
```

Regras de nomeação:

- Usar minúsculas e hífens: `comixflix`, `portal-clientes`, `gestao-estoque`.
- Evitar espaços, acentos, caracteres especiais e nomes genéricos.
- Não incluir versão no nome da pasta.

Não inicialize Git, não crie aplicação e não monte manualmente a árvore de diretórios técnica nesta etapa. Essas atividades serão realizadas pelo Antigravity Kit.

Etapa 2 — Criar a diretiva de design

Antes de executar o Antigravity, criar somente o diretório de diretivas de design necessário para armazenar os artefatos de elaboração.

Código de referência

```
mkdir -p documentation/directives/design
```

Nenhuma outra estrutura de diretórios deve ser criada manualmente neste guia. O Antigravity Kit deverá criar, ajustar ou complementar a estrutura do projeto.

Etapa 3 — Criar os arquivos de diretiva iniciais

Criar inicialmente os arquivos vazios que serão preenchidos e gerados durante o fluxo.

Código de referência

```
touch documentation/directives/0.ideia-inicial.md
touch documentation/directives/1.ideia-projeto.md
touch documentation/directives/5.projeto.md
touch documentation/directives/design/1.ideia-design.md
touch documentation/directives/design/5.design.md
```

Arquivos inicialmente vazios:

Arquivo	Finalidade
`documentation/directives/0.ideia-inicial.md`	Ideia bruta, problema, público e contexto inicial
`documentation/directives/1.ideia-projeto.md`	Especificação inicial de produto e tecnologia
`documentation/directives/5.projeto.md`	Especificação técnica e funcional definitiva
`documentation/directives/design/1.ideia-design.md`	Ideia e requisitos iniciais de UI/UX
`documentation/directives/design/5.design.md`	Especificação definitiva de UI/UX e design system

Também copiar ou criar os templates de instrução e prompts utilizados pela organização:

Código de referência

```
touch documentation/directives/0.prompt-iaexterna-inicial.md
touch documentation/directives/2.instrucao-projeto.md
touch documentation/directives/3.prompt-iaexterna-projeto.md
touch documentation/directives/4.prompt-antigravity.md
touch documentation/directives/design/2.instrucao-design.md
touch documentation/directives/design/3.prompt-iaexterna-design.md
touch documentation/directives/design/4.prompt-stitch-design.md
```

Etapa 4 — Criar e configurar o ambiente

Criar `.env` na raiz do projeto. Este arquivo é local e nunca deve ser versionado.

Código de referência

```
touch .env
```

Criar também `.env-example`, que pode ser versionado e deve listar as variáveis exigidas sem valores sensíveis.

Código de referência

```
touch .env-example
```

Modelo inicial de `.env-example`:

Código de referência

```
# Aplicação
NODE_ENV=development
NEXT_PUBLIC_APP_NAME=
NEXT_PUBLIC_APP_URL=http://localhost:3000

# GitHub: somente servidor/local, nunca NEXT_PUBLIC_
GITHUB_TOKEN=
GITHUB_OWNER=
GITHUB_REPOSITORY=

# Feature flags
NEXT_PUBLIC_ENABLE_LIGHT_THEME=true
NEXT_PUBLIC_ENABLE_ANALYTICS=false

# MCPs e integrações externas
MCP_GOOGLE_STITCH_ENABLED=false
MCP_GITHUB_ENABLED=false
MCP_FIREBASE_ENABLED=false
MCP_SUPABASE_ENABLED=false
GOOGLE_STITCH_API_KEY=
SUPABASE_SERVICE_ROLE_KEY=
```

A lista final de variáveis será atualizada após a geração de `5.projeto.md`, pois depende das integrações e da arquitetura aprovadas.

Etapa 5 — Configurar o GitHub Personal Access Token

Inserir o GitHub Personal Access Token somente no `.env` local.

Código de referência

```
GITHUB_TOKEN=github_pat_substitua_por_seu_token
GITHUB_OWNER=seu-usuario-ou-organizacao
GITHUB_REPOSITORY=nome-do-repositorio
```

Regras obrigatórias:

- Nunca usar ``NEXT_PUBLIC_GITHUB_TOKEN``.
- Nunca incluir o token em arquivos versionados.
- Nunca escrever o token em URL de repositório, script rastreado, README, prompt, documentação pública ou log.
- Conceder apenas permissões necessárias.
- Preferir token com expiração e política de renovação.
- Revogar e substituir imediatamente em caso de exposição.
- Em CI/CD, usar secrets do provedor; não enviar o `.env`` local para pipelines ou serviços externos.
- Registrar finalidade, responsável e data de revisão do token, sem registrar seu valor.

Uso local seguro, quando necessário:

Código de referência

```
set -a
source .env
set +a
```

Não utilizar comandos que imprimam `GITHUB_TOKEN`, dumps de ambiente ou logs de depuração que exponham variáveis.

Etapa 6 — Executar Antigravity Kit

Na raiz do projeto, executar:

Código de referência

```
ag-kit init
```

O Antigravity Kit deve ser responsável por inicializar ou complementar:

- Repositório Git e arquivos de controle.
- Aplicação base e dependências essenciais.
- Estrutura técnica de diretórios.
- Estrutura completa de ``documentation/``.
- Arquivos operacionais.
- Lint, testes, convenções e configuração inicial.
- Automação e integrações suportadas pelo kit.

Após a execução:

1. Confirmar que os arquivos em ``documentation/directives/`` foram preservados.
2. Confirmar que `.env`` permanece local e não foi rastreado.

3. Confirmar que `.env-example` não contém valores secretos.
 4. Comparar arquivos criados com as diretivas existentes.
 5. Registrar conflitos ou alterações necessárias.
 6. Confirmar que a estrutura criada atende à documentação que será gerada em `5.projeto.md`.
- Não recriar manualmente uma estrutura que o kit já estabeleceu.

Etapa 7 — Preencher a ideia inicial

Preencher `documentation/directives/0.ideia-inicial.md` usando linguagem de negócio.

O arquivo deve responder:

- Qual problema existe?
- Quem é afetado?
- Qual solução é proposta?
- Qual é o objetivo principal?
- Qual resultado significa sucesso?
- Quais funcionalidades parecem essenciais?
- Quais integrações, dados, restrições e referências são conhecidos?
- O que é confirmado e o que é hipótese?

Modelo mínimo:

Código de referência

```
# Ideia inicial

## Nome provisório

## Problema

## Público

## Solução proposta

## Diferencial esperado

## Funcionalidades imaginadas

## Restrições conhecidas

## Integrações e fontes de dados

## Hipóteses a validar

## Referências visuais e concorrentes
```

Etapa 8 — Gerar a ideia de projeto

Usar como entrada `0.ideia-inicial.md`, o prompt inicial e as restrições conhecidas. Gerar ou preencher `documentation/directives/1.ideia-projeto.md`.

Registrar contexto, requisitos, módulos, dados, integrações, restrições, hipóteses, lacunas, MVP e evoluções. Revisar se a ideia original foi preservada.

Etapa 9 — Gerar o projeto definitivo

Usar `1.ideia-projeto.md`, `2.instrucao-projeto.md` e `3.prompt-iaexterna-projeto.md`, quando aplicável. Gerar `documentation/directives/5.projeto.md`.

O documento deve incluir visão, escopo, requisitos testáveis, regras, aceitação, arquitetura, diagramas, dados, APIs, integrações, segurança, privacidade, observabilidade, testes, implementação, roadmap, deploy, rollback, backup, operação, riscos, matriz de rastreabilidade e regras de ambiente.

Aprovar apenas se o MVP estiver separado de evoluções, credenciais não estiverem incluídas, integrações tiverem fallback, o token GitHub estiver previsto como variável privada e a documentação oficial estiver em `documentation/`.

Etapa 10 — Gerar a ideia de design

Usar `5.projeto.md` para preencher `documentation/directives/design/1.ideia-design.md` com objetivos de UX, usuários, jornadas, telas, arquitetura de informação, referências visuais, breakpoints, estados, restrições e componentes.

Etapa 11 — Gerar o design definitivo

Usar `design/1.ideia-design.md`, `design/2.instrucao-design.md` e `design/3.prompt-iaexterna-design.md`. Gerar `design/5.design.md` com design system, componentes, estados, wireframes, responsividade, acessibilidade, motion, conteúdo, performance, código de referência, ambiente, feature flags e estados de integração.

Etapa 12 — Revisão cruzada

Revisar os cinco documentos de diretiva:

- ``0.ideia-inicial.md``.
- ``1.ideia-projeto.md``.
- ``5.projeto.md``.
- ``design/1.ideia-design.md``.
- ``design/5.design.md``.

Verificar escopo, entidades, fluxos, integrações, token GitHub, segurança, estados, acessibilidade, operação e documentação. Resolver divergências antes de iniciar o código.

Etapa 13 — Atualizar ambiente e documentação oficial

Após aprovar `5.projeto.md` e `5.design.md`:

1. Atualizar ``env-example`` com todas as variáveis definitivas.
2. Garantir que ``env`` contém somente valores do ambiente local.
3. Documentar variáveis sem segredos em ``documentation/operations/environment.md``.
4. Usar a estrutura criada pelo Antigravity para materializar a documentação oficial.

5. Criar documentos com conteúdo real.
6. Manter versões, responsáveis, status, revisão e links relacionados.

Etapa 14 — Congelar o escopo de início

Confirmar MVP, funcionalidades adiadas, dependências, riscos, fontes autorizadas, autenticação, ambientes, variáveis obrigatórias, token GitHub com menor privilégio, critérios de qualidade, critérios de lançamento e backlog priorizado.

Mudanças posteriores devem registrar impacto em requisitos, arquitetura, design, testes, prazo, operação e documentação.

Etapa 15 — Checkpoint para iniciar desenvolvimento

Base e ferramentas

- [] Pasta do projeto criada.
- [] Diretório `documentation/directives/design` criado.
- [] Arquivos de diretiva iniciais criados.
- [] Antigravity Kit executado com `ag-kit init`.
- [] Estrutura gerada pelo Antigravity revisada.
- [] Arquivos de diretiva preservados.

Ambiente e GitHub

- [] `.env` criado e não versionado.
- [] `.env-example` criado, versionado e sem segredos.
- [] `GITHUB_TOKEN` está apenas no `.env` local ou secret remoto.
- [] `GITHUB_TOKEN` não tem prefixo `NEXT_PUBLIC_`.
- [] Token possui permissões mínimas, expiração e responsável.
- [] `GITHUB_OWNER` e `GITHUB_REPOSITORY` definidos quando necessários.
- [] Não há segredos em arquivos, logs ou prompts.
- [] Variáveis obrigatórias possuem validação prevista.

Especificações

- [] `0.ideia-inicial.md` preenchido.
- [] `1.ideia-projeto.md` preenchido.
- [] `5.projeto.md` preenchido e revisado.
- [] `design/1.ideia-design.md` preenchido.
- [] `design/5.design.md` preenchido e revisado.
- [] Prompts e instruções preservados.
- [] MVP, evolução e fora de escopo separados.

- [] Riscos, premissas e pontos em aberto registrados.

Arquitetura, design e operação

- [] Modelo de dados revisado.
- [] Requisitos são verificáveis.
- [] Integrações têm configuração, limites, responsável e fallback.
- [] Segurança, privacidade e autorização definidas.
- [] Estratégia de testes existe.
- [] Observabilidade, backup e rollback previstos.
- [] Documentação oficial planejada.

Autorização

- [] Backlog inicial priorizado.
- [] Critérios de pronto definidos.
- [] Primeira fatia vertical selecionada.
- [] Responsáveis técnicos definidos.
- [] Desenvolvimento autorizado.

Primeira atividade após aprovação

A primeira implementação deve ser uma fatia vertical pequena, testável e demonstrável. Ela deve ler configurações com validação, conectar o ambiente de dados, entregar um fluxo simples do MVP, incluir tratamento de erro, telemetria e testes, atualizar a documentação e passar nos quality gates.

Exemplo: dados de fixture → listagem acessível → detalhe → teste E2E. Depois, substituir a fixture pela primeira integração autorizada.

Manutenção do guia

Revisar este guia quando houver alteração no Antigravity Kit, no fluxo de diretivas, na estratégia de ambiente, no uso de token GitHub, no padrão documental ou no processo de aprovação.